Objetivo

Definir la arquitectura tecnológica para una solución de videollamadas basada en WebRTC, con soporte para aplicaciones web y móviles, comunicación de audio y video en tiempo real y capacidad para compartir pantalla.
La solución deberá permitir una interfaz gráfica y flujos de negocio completamente personalizados, restringir el acceso web a dominios autorizados y soportar una concurrencia inicial estimada entre 50 y 200 sesiones simultáneas.

Arquitectura de infraestructura propuesta

La solución se organizará en cuatro capas principales:

Capa 1. Clientes

- Landing web autorizado.
- Aplicaciones móviles.
- Interfaz y flujos de negocio personalizados.

Capa 2. Acceso y backend

- Validación del dominio autorizado para el acceso web.
- Autenticación y autorización.
- Gestión de llamadas.
- Emisión de credenciales temporales.
- Backend en Go

Capa 3. Servicio de medios WebRTC

- Gestión de señalización y conexiones.
- Procesamiento y retransmisión de audio y video.
- Compartir pantalla.
- Plataforma por seleccionar: LiveKit, Jitsi, mediasoup o Janus.
- Servicio TURN para conexiones que requieran retransmisión.

Capa 4. Servicios de soporte

- PostgreSQL: persistencia de datos.
- Dragonfly: almacenamiento en memoria y coordinación, sujeto a validar compatibilidad.

- Monitorización, logs y alertas.
- Seguridad de red, certificados TLS y gestión de secretos.

Distribución propuesta

Infraestructura corporativa: backend, PostgreSQL y Dragonfly.

Infraestructura cloud: servidor WebRTC y TURN.

Capacidad inicial

Entre 50 y 200 sesiones simultáneas, equivalentes a un máximo estimado de 400 participantes conectados.

El dimensionamiento definitivo se determinará mediante pruebas de carga.

Seguridad y control de acceso

Se definió como requisito restringir el acceso web a dominios autorizados, incluyendo la evaluación de estas medidas:

- Lista de orígenes web permitidos y validación de solicitudes.
- Control de conexiones WebSocket según la plataforma.
- Emisión de credenciales temporales desde el backend.
- Permisos específicos para las salas y las operaciones multimedia.
- Protección de las claves maestras y secretos.
- HTTPS/WSS y cifrado de medios WebRTC.
- Firewall y servicios de datos limitados a redes privadas.
- Configuración segura de TURN.
- Pruebas de acceso directo, credenciales inválidas y vencidas.

Limitación importante: restringir el encabezado `Origin` no impide por sí solo que un cliente externo intente conectarse, porque puede falsificarlo. Además, las aplicaciones móviles nativas no dependen necesariamente de un origen web. Por eso, aunque la restricción de origen se mantiene como requisito, cada plataforma deberá evaluarse para verificar cómo proteger el acceso directo al servicio multimedia.

Modelo de la landing

La landing no usa la interfaz prefabricada de LiveKit. La pantalla, el flujo de entrada y los controles son propios. El SDK de LiveKit, en el navegador, solo conecta la sala, publica las pistas y entrega los flujos remotos para pintarlos en esa interfaz.

Apagar la cámara, silenciar el micrófono o dejar de compartir pantalla llama al SDK. Eso detiene el envío. LiveKit deja de retransmitir. El backend no participa en cada control.

Flujo de la llamada

1. El usuario entra por un dominio autorizado.
2. La landing pide al backend autorización para esa llamada.
3. El backend devuelve un token de corta duración, la sala y los permisos (publicar audio, video o pantalla, o solo ver).
4. La landing conecta el SDK con ese token.
5. Según el permiso, publica cámara y micrófono o entra solo como espectador.
6. Los demás clientes reciben el cambio por el WebSocket de señalización y actualizan su propia pantalla.

El audio, el video y la pantalla viajan por WebRTC, no por ese WebSocket.

Contrato con el backend

La landing solo consume tres cosas: la autorización de entrada, el token (identidad, sala, caducidad y permisos) y el aviso de que la llamada ya no es válida. Crear la sala, cerrarla, firmar el token y auditar no forman parte del frontend.

Si el token caduca a mitad de la llamada, la landing pide uno nuevo al backend y reconecta. No lo renueva por su cuenta.

Estados que debe cubrir la interfaz

- Antesala: vista previa, elección de cámara y micrófono, unirse.
- Permiso denegado o dispositivo ausente.
- Conectando, en llamada y reconectando.
- Participante unido, con cámara apagada, en silencio o compartiendo pantalla.
- Calidad de red degradada.
- Token vencido, llamada cerrada o acceso rechazado.
- Salida de la sala.

Cada estado visual sale de un evento del SDK o de una respuesta del backend. Ocultar un `<video>` sin dejar de publicar la pista no cuenta como apagar la cámara.

Límites del navegador

La cámara, el micrófono y la pantalla solo funcionan en un contexto seguro (HTTPS). El usuario puede rechazar el permiso, y la landing tiene que poder continuar o explicar el bloqueo.

Compartir pantalla es otra pista, distinta de la cámara, y el navegador muestra su propio diálogo. En Safari hay que validar autoplay, permisos y pantalla compartida aparte de Chrome.

Distribución de responsabilidades

Componente	Responsabilidad principal
Landing web	Interfaz, flujo de entrada y comunicación con el backend.
Aplicaciones móviles	Interfaz, permisos de cámara y micrófono, llamadas y captura de pantalla.
Backend	Autorización, gestión de llamadas y emisión de credenciales temporales.
Servidor WebRTC	Señalización y procesamiento o retransmisión de audio y video.
TURN	Retransmisión de medios cuando la conexión directa no sea posible.
PostgreSQL	Persistencia de usuarios, llamadas, estados y auditoría.
Dragonfly	Datos temporales y coordinación compartida, si la plataforma seleccionada lo requiere y se valida la compatibilidad.
Infraestructura	Redes, firewall, DNS, TLS, secretos y disponibilidad.

Monitorización	Métricas, logs, alertas y diagnóstico de incidentes.

Puertos y reglas de firewall

Los puertos siguientes son los valores de referencia de LiveKit autogestionado. Deben confirmarse con la configuración definitiva y las reglas del proveedor de nube.

Puerto / protocolo	Uso	Regla recomendada
TCP 443	HTTPS y WebSocket seguro	Público, mediante balanceador o proxy.
TCP 7880	API y señalización LiveKit	Solo interno detrás del balanceador.
TCP 7881	Transporte ICE/TCP alternativo	Público cuando se habilite.
UDP 50000–60000	Medios WebRTC	Público hacia los nodos LiveKit.
UDP 3478	TURN/UDP, si se habilita	Público hacia el servidor TURN.
TCP 5349	TURN/TLS, si se habilita	Público, con certificado válido.
UDP 49152–65535	Relés de coturn, si se utiliza esa configuración	Público hacia coturn; ajustar el rango configurado.

TCP 5432	PostgreSQL	Solo desde los servicios autorizados.
TCP 6379	Redis	Solo desde los nodos que lo necesitan.

Infraestructura actualizada

Componente	Función	Ubicación propuesta
Backend de negocio	Autorización, gestión de llamadas y emisión de tokens	Infraestructura corporativa
PostgreSQL	Persistencia de usuarios, llamadas y auditoría	Infraestructura corporativa
Dragonfly	Coordinación compartida de LiveKit y datos temporales compatibles	Red privada accesible solo por los nodos autorizados
LiveKit	Señalización y procesamiento de audio y video	Cloud
TURN	Retransmisión de medios cuando la conexión directa falla	Cloud
Landing web y aplicaciones móviles	Interfaz y comunicación con los servicios	Clientes web y móviles

Monitorización	Métricas, logs y alertas	Según la infraestructura disponible

Comparación técnica para 50–200 llamadas simultáneas

Criterio	Python + FastAPI	Go + `fiber`
Generar tokens LiveKit	Soportado	Soportado
Crear y cerrar salas	API del SDK	API del SDK
Integrar autenticación existente	Mediante bibliotecas y middleware	Mediante bibliotecas y middleware
Concurrencia de solicitudes	`asyncio` y servidor ASGI	Goroutines y servidor HTTP
Trabajo intensivo de CPU	Puede requerir procesos o workers adicionales	Puede ejecutar tareas concurrentes, aunque también requiere controlar tareas costosas
Despliegue	Entorno Python y dependencias	Binario compilado y dependencias de ejecución generalmente reducidas
Operación y mantenimiento	Depende de experiencia del equipo	Depende de experiencia del equipo

Plataformas WebRTC

Criterio	LiveKit	Jitsi	mediasoup	Janus
Tipo de solución	Plataforma WebRTC con SFU y señalización integrada	Plataforma de videoconferencia completa	SFU de bajo nivel	Gateway WebRTC extensible mediante plugins
Personalización de interfaz	Alta	Media-alta, con adaptación	Alta	Alta
Control de señalización y medios	Medio-alto	Medio	Alto	Alto
Complejidad de integración	Media	Media	Alta	Alta
Desarrollo de lógica propia	Moderado	Moderado	Alto	Alto
Licencia principal	Apache 2.0	Componentes principales mayoritariamente Apache 2.0	ISC	GPLv3
Compatibilidad web y móvil	SDK disponibles; validar	Integración web y SDK móviles; verificar cada cliente	Cliente web oficial y opciones	API web y opciones de cliente

	funciones por plataforma	comunitarias para móvil	comunitarias para móvil

Costos de uso

Servidor	Licencia del software	Costos de operación	Complejidad de integración
LiveKit	Código abierto para alojarlo en infraestructura propia.	Servidores, transferencia de datos y TURN si se requiere.	Media. Incluye componentes para salas, señalización y SDK.
Jitsi Meet	Código abierto y autohospedable.	Servidor, ancho de banda y TURN.	Baja-media si se aprovecha su interfaz; mayor si se personaliza profundamente.
mediasoup	Código abierto.	Servidores, tráfico y componentes adicionales que debes operar.	Alta. Requiere desarrollar gran parte de la señalización y la lógica de salas.
Janus	Código abierto, licencia GPLv3 para el servidor.	Servidores, tráfico y servicios auxiliares.	Media-alta. Debes integrar sus plugins y protocolos con tu aplicación.

Documentación

LiveKit: https://docs.livekit.io/reference/

Jitsi Meet:https://jitsi.github.io/handbook/docs/category/user-guide

Mediasoup: https://mediasoup.org/documentation/v3

Janus: https://janus.conf.meetecho.com/docs

Escalabilidad

Criterio	LiveKit	Jitsi Meet	mediasoup	Janus
Tipo de solución	Servidor SFU integrado	Plataforma de videoconferencia	Biblioteca SFU para Node.js	Pasarela WebRTC modular
Escalado horizontal	Sí, con Redis y varios nodos	Sí, agregando Videobridges	Sí, distribuyendo workers y routers	Sí, mediante una arquitectura distribuida propia
Distribución de llamadas entre nodos	Gestionada por el servidor	Gestionada mediante sus componentes de conferencias	Debe implementarla la aplicación	Debe diseñarse e implementarse
Escalado de una misma llamada entre nodos	Limitado: una sala debe caber en un nodo	Existen mecanismos de distribución entre bridges, según la configuración	Posible conectando routers, con complejidad adicional	Depende de la arquitectura y los plugins implementados
Balanceo de nuevas sesiones	Integrado en modo distribuido	Integrado en la arquitectura de conferencias	Lógica de asignación propia	Lógica de asignación propia

Alta disponibilidad	Requiere redundancia y estrategia de recuperación	Requiere redundancia de los componentes centrales y de medios	Requiere redundancia y recuperación desarrolladas	Requiere redundancia y recuperación desarrolladas
Autoescalado	Posible con infraestructura y métricas adecuadas	Posible agregando o retirando Videobridges	Posible con orquestación y lógica propia	Posible con orquestación y lógica propia
Adecuación inicial a 50–200 llamadas 1:1	Alta	Alta	Media-alta, si se acepta el desarrollo adicional	Media, si se acepta el desarrollo adicional

Limitantes

LiveKit

Limitaciones

- Una sala debe ejecutarse en un solo nodo; el límite depende de sus recursos y de la carga multimedia.
- Redis es necesario para el modo distribuido.
- La infraestructura WebRTC requiere acceso de red adecuado; en Kubernetes, la configuración habitual limita cada nodo a un pod de LiveKit.
- Si falla el nodo que aloja una llamada, los participantes pueden tener que reconectarse. La redundancia no garantiza continuidad transparente.

Jitsi Meet

Limitaciones

- Requiere operar varios componentes, entre ellos Prosody, Jicofo y Jitsi Videobridge.
- Escalar los Videobridges no elimina la necesidad de mantener disponibles los componentes centrales.
- Personalizar profundamente la experiencia puede implicar modificar el cliente o desarrollar una integración adicional.
- Debe probarse el comportamiento cuando se retira o pierde un Videobridge.

mediasoup

Limitaciones

- No incluye una plataforma completa de gestión de usuarios, autenticación y señalización de llamadas.
- Cada worker de medios utiliza un solo núcleo de CPU; hay que distribuir las salas entre workers y máquinas.
- Conectar routers de diferentes workers o servidores requiere implementar la lógica de distribución correspondiente.
- No decodifica ni transcodifica los medios por defecto: reenvía paquetes RTP.

Janus

Limitaciones

- Es una pasarela WebRTC, no una solución integral de videollamadas con toda la lógica de negocio.
- La distribución de sesiones, el enrutamiento entre instancias y la gestión del estado deben resolverse mediante la arquitectura de la aplicación y los componentes elegidos.
- Los plugins tienen funciones diferentes; no todos resuelven los mismos casos de uso.
- El equipo debe implementar y probar la recuperación cuando una instancia deja de estar disponible.

Consumo de recursos

Recurso	LiveKit	Jitsi Meet	mediasoup	Janus
CPU	Media-alta, según cantidad de flujos y paquetes	Media-alta; depende de Videobridge y tráfico	Media-alta; cada worker utiliza un núcleo	Media-alta; depende de plugins y tráfico

RAM	Media; depende de salas, conexiones y configuración	Media; JVM y componentes adicionales	Media; workers, routers y estado de conexiones	Media; sesiones, plugins y buffers
Ancho de banda	Alto con video activo	Alto con video activo	Alto con video activo	Alto con video activo
Disco para llamadas sin grabación	Bajo	Bajo	Bajo	Bajo
Costo computacional adicional por transcodificación	Si se agregan servicios que transcodifican	Si se agregan servicios que transcodifican	No transcodifica medios por defecto	Depende del plugin y de la solución implementada
Aprovechamiento de múltiples núcleos	Sí	Sí	Mediante varios workers	Depende de cómo se distribuyan las sesiones y los procesos
Referencia oficial de rendimiento	Sí, con pruebas de carga publicadas	Sí, con evaluación de Videobridge	Documentación de escalabilidad, sin una cifra universal	Sin una cifra universal aplicable a tu escenario